Do you know

https://github.com/godotengine/godot

Yes 👍

The repository you linked — Godot Engine on GitHub — is the **official source code of the Godot game engine**.

What it is

- A **free and open-source game engine** used to create **2D and 3D games**.
- It supports exporting games to **Windows, macOS, Linux, Android, iOS, Web, and more**.
- The whole engine is released under the **MIT license**, meaning you can use it commercially without paying royalties. GitHub

What the GitHub repo contains

The repository you shared:

- The **entire engine source code** (mostly written in **C++**).
- Build scripts and platform support.
- Issues and pull requests where developers contribute improvements.
- Over **100k+ stars and thousands of contributors**, showing it's a very active open-source project. GitHub

Who created it

The engine was originally developed by **Juan Linietsky** and **Ariel Manzur**, then open-sourced in 2014 and is now maintained by the community and the **Godot Foundation**. GitHub

Why developers like it

- Completely **free (no revenue cut)** 💰
- **Lightweight editor** compared with some engines
- Uses **GDScript** (a Python-like language) plus **C#, C++, and others**

- Strong **2D support**, good for indie games

Example games made with it

Some known games built with Godot include:

- Cassette Beasts
- Dome Keeper
- Brotato

✅ If you want, I can also show you:

- **How the Godot engine source code is structured** (good if you're reading the repo)
- **How to build the engine from that GitHub repo**
- Or **how beginners start making a game with Godot**.

Sources

Great.

I want you to find the file that resolves physics collisions

Great. I want you to find the file that resolves 2D physics collisions.

Yes — in the Godot Engine source code, **2D physics collision resolution is mainly implemented inside the 2D physics module**. The key file you're looking for is:

Main collision resolution file

- `modules/godot_physics_2d/godot_space_2d.cpp`

This file is the **core of the 2D physics simulation step**. It handles:

- Detecting contacts between bodies
- Building **contact manifolds**
- Solving **collision impulses**
- Integrating physics each frame

In particular, the solver logic runs during the **space step**, where bodies are iterated and collisions are resolved.

Typical flow in that file:

```
Space2D::step()
   -> broadphase collision detection
   -> narrowphase collision detection
   -> contact creation
   -> constraint solving (collision response)
```

Important related files (also part of collision resolution)

1️⃣ Space simulation

- `modules/godot_physics_2d/godot_space_2d.cpp`
- `modules/godot_physics_2d/godot_space_2d.h`

Handles:

- Physics step
- Collision pair processing
- Solver loops

2️⃣ Collision detection (shapes)

- `modules/godot_physics_2d/godot_collision_solver_2d.cpp`

Handles:

- Shape-vs-shape collision tests
- Contact generation
- Separating axis / GJK style checks

3️⃣ Body physics

- `modules/godot_physics_2d/godot_body_2d.cpp`

Handles:

- Applying impulses
- Velocity integration
- Body properties

4️⃣ Contact constraints

- `modules/godot_physics_2d/godot_constraint_2d.cpp`

Handles:

- Contact constraints
- Friction
- restitution (bouncing)

✅ **If you want the exact collision response code**, look inside:

```
modules/godot_physics_2d/godot_space_2d.cpp
```

and search for functions like:

```
solve_constraints()
_step()
integrate_forces()
```

These contain the **actual impulse solver that resolves 2D collisions**.

💡 **Tip if you're studying the engine:**

The easiest entry point is the **physics step pipeline**:

```
PhysicsServer2D
   ↓
Space2D::step()
   ↓
CollisionSolver2D
   ↓
Constraint solver
```

✅ If you want, I can also show you:

- The **exact 20–30 lines where the impulse collision resolution happens**
- Or a **map of the whole Godot 2D physics pipeline** (which makes reading the repo *much* easier).

I am specifically looking for code relevant to resolutions between

staticbody2d and characterbody2d

For **collisions between** `CharacterBody2D` **and** `StaticBody2D`, the important thing to understand in the Godot Engine architecture is:

👉 **CharacterBody2D collisions are *not* resolved by the general physics solver.**
They are resolved by **kinematic motion code**, because `CharacterBody2D` is a **kinematic body** controlled manually via `move_and_slide()` / `move_and_collide()`.

So the relevant code is **not mainly inside the physics constraint solver**, but inside the **character motion implementation**.

---

1. The main file you want

The core implementation is in:

```
scene/2d/physics/character_body_2d.cpp
```

This file contains the implementation of:

```
CharacterBody2D::move_and_slide()
CharacterBody2D::move_and_collide()
```

These functions perform:

1. **Motion sweep test**

2. **Collision detection**
3. **Slide / depenetration resolution**
4. **Velocity projection along surfaces**

This is the code path used when a **CharacterBody2D hits a StaticBody2D**.

---

2. The actual collision query

Inside `move_and_collide`, the engine calls the physics server:

```
PhysicsServer2D::body_test_motion()
```

That happens through:

```
PhysicsServer2D::body_test_motion
```

Implementation is inside:

```
servers/physics_2d/physics_server_2d_sw.cpp
```

and the internal solver:

```
servers/physics_2d/space_2d_sw.cpp
```

This performs the **swept collision test against StaticBody2D shapes**.

---

3. Where the slide resolution happens

Inside `character_body_2d.cpp`, after collision detection:

Typical code structure:

```
for (int i = 0; i < max_slides; i++) {
    PhysicsServer2D::MotionResult result;

    bool collided = PhysicsServer2D::body_test_motion(...);
```

```
    if (!collided) {
        position += motion;
        break;
    }

    Vector2 normal = result.collision_normal;
```

‹/› C++

This is the **actual collision resolution logic** between:

```
CharacterBody2D  -> moving
StaticBody2D     -> blocking surface
```

The important operation is:

```
motion.slide(normal)
```

which removes the velocity component along the collision normal.

---

4. Static bodies themselves

`StaticBody2D` code lives here:

```
servers/physics_2d/body_2d_sw.cpp
```

But they **do not resolve collisions**.

They only provide **shapes for queries**.

All motion resolution is done by the **moving body** (the character).

---

5. The full pipeline

When `CharacterBody2D` hits `StaticBody2D`:

```
CharacterBody2D::move_and_slide()
        |
        ▼
move_and_collide()
```

```
        |
        ▼
PhysicsServer2D::body_test_motion()
        |
        ▼
Space2DSW collision sweep
        |
        ▼
MotionResult returned
        |
        ▼
slide / depenetration logic
```

---

6. The exact files involved

Most relevant:

```
scene/2d/physics/character_body_2d.cpp
scene/2d/physics/character_body_2d.h
```

Supporting physics:

ChatGPT Log in Sign up for free

```
servers/physics_2d/space_2d_sw.cpp
servers/physics_2d/body_2d_sw.cpp
```

✅ **If your goal is understanding the collision resolution math**, the most important code to read is:

```
CharacterBody2D::_move_and_slide()
```

inside

```
scene/2d/physics/character_body_2d.cpp
```

💡 If you want, I can also show you:

- The **exact 40–60 lines where the sliding collision resolution happens**
- The **function in** `space_2d_sw.cpp` **that computes the collision normal**
- Or a **diagram of how Godot's 2D physics engine is structured internally** (which helps a lot when navigating the repo).

Fantastic, can you explain collsiion resolution for static body 2d to and rigid body 2d?

For **collisions between** `RigidBody2D` **and** `StaticBody2D`, the process is different from `CharacterBody2D`.
Here the collision **is handled by the physics solver** inside the Godot physics engine.

`RigidBody2D` participates in the **constraint-based physics simulation**, while `StaticBody2D` acts as an **immovable collider**.

I'll walk through the actual pipeline in the source.

---

1. Where the resolution code lives

Main files involved:

```
modules/godot_physics_2d/godot_space_2d.cpp
modules/godot_physics_2d/godot_body_2d.cpp
modules/godot_physics_2d/godot_constraint_2d.cpp
modules/godot_physics_2d/godot_collision_solver_2d.cpp
```

The most important file for **resolution** is:

```
modules/godot_physics_2d/godot_constraint_2d.cpp
```

This contains the **impulse solver** used to resolve contacts.

---

2. High-level physics step

Each frame the physics engine runs something like:

```
Space2D::step()
```

Inside:

```
integrate forces
broadphase collision detection
narrowphase collision detection
build contact constraints
solve constraints
integrate velocities
```

That function lives in:

```
modules/godot_physics_2d/godot_space_2d.cpp
```

---

3. Collision detection

First the engine determines which shapes overlap.

Broadphase:

```
BVH / AABB tree overlap detection
```

Then narrowphase:

```
godot_collision_solver_2d.cpp
```

This computes:

```
contact points
penetration depth
collision normal
```

Example result:

```
RigidBody2D
    ↓
contact normal
    ↓
StaticBody2D
```

---

4. Creating the contact constraint

When a rigid body hits a static body, the engine creates a **contact constraint**.

Conceptually:

```
RigidBody2D velocity
        ↓
Collision normal
        ↓
Impulse applied
```

Static body properties:

```
mass = infinite
velocity = 0
```

So **only the rigid body moves**.

---

5. The impulse solver

In `godot_constraint_2d.cpp` the solver computes an impulse:

Simplified physics equation:

```
J = -(1 + e) * (v · n)
     --------------------
       inverse_mass
```

Where:

```
J = impulse
e = restitution (bounciness)
```

```
v = relative velocity
n = collision normal
```

Because StaticBody has:

```
inverse_mass = 0
```

the impulse is applied entirely to the rigid body.

---

6. Velocity update

The rigid body's velocity changes:

```
v' = v + J * inverse_mass * normal
```

Angular velocity also changes if the contact point is offset:

```
w' = w + I⁻¹ (r × J)
```

Where:

```
I⁻¹ = inverse inertia
r   = vector from center of mass
```

This is implemented in:

```
godot_body_2d.cpp
```

---

7. Position correction (depenetration)

If bodies overlap, the solver pushes them apart using **bias correction**.

Typical formula:

```
correction = penetration_depth * bias_factor
```

Then the rigid body position is shifted.

Static body stays fixed.

---

8. Friction resolution

After the normal impulse, the solver computes **tangent friction**.

```
tangent = velocity - normal_component
```

Then friction impulse:

```
Jt = -vt / effective_mass
```

Clamped by:

```
|Jt| <= μ * J
```

Where:

```
μ = friction coefficient
```

---

9. Full resolution pipeline

For `RigidBody2D` hitting `StaticBody2D`:

```
Space2D::step()
        │
        ▼
Broadphase detection
        │
        ▼
CollisionSolver2D
        │
        ▼
Contact constraints created
        │
        ▼
Impulse solver
        │
        ▼
```

```
RigidBody velocity updated
        |
        ▼
Position correction
```

---

10. Key conceptual difference

Body type	Collision resolution
CharacterBody2D	manual slide solver
RigidBody2D	impulse constraint solver
StaticBody2D	immovable collider

So:

```
RigidBody2D ← impulse solver
StaticBody2D ← infinite mass surface
```

✅ Short summary

When a `RigidBody2D` collides with a `StaticBody2D`:

1. Engine detects contact.
2. Contact constraint is created.
3. Impulse is calculated using collision normal and velocity.
4. Impulse is applied to the rigid body.
5. Static body remains stationary.

If you'd like, I can also show you something very useful when reading the engine:

- **The exact ~80 lines of Godot's impulse solver**
- **How Godot handles stacked rigid bodies**
- **Why rigid bodies sometimes jitter against static surfaces** (a very interesting detail in the solver).